

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Computing Cross Entropy in more than one way

Application: Implementing sets as lists

Applying Recursion: Polygon Triangulation Problem

Four solutions for Fibonacci

Derangements

What is Cross Entropy?

Definition (Cross Entropy)

The **cross entropy** $H(t, p)$ of a probability distribution p relative to the distribution t over a given set S is defined as:

$$H(t, p) = - \sum_{x \in S} t(x) \log_2 p(x)$$

Remark

It measures the difference between two probability distributions (t =true, p =predicted). It is used in Deep Learning as a loss function.

Remark

We assume no element of p is 0.

Computing Cross Entropy: the very first attempt

Example (Basic recursion using `if-then-else`)

```
crossEnt :: Floating t => [t] -> [t] -> t
crossEnt ts ps =
  if (null ts) then 0
  else ((-1) * (head ts) * (logBase 2 (head ps))
        + crossEnt (tail ts) (tail ps))
```

Computing Cross Entropy, take 1

Example (Basic recursion using guards)

```
crossEnt :: Floating t => [t] -> [t] -> t
crossEnt ts ps
  | null ts      = 0
  | otherwise = (-1) * (head ts) *
                  (logBase 2 (head ps))
                  + crossEnt (tail ts) (tail ps)
```

Computing Cross Entropy, take 2

Example (Pattern matching based equations)

`crossEnt [] [] = 0`

`crossEnt (t:ts) (p:ps) = (-1) * t * logBase 2 p
+ crossEnt ts ps`

Computing Cross Entropy, take 3

Example (Tail recursion)

```
crossEnt ps qs = cea ps qs 0
  where
    cea [] [] a = a
    cea (p:ps) (q:qs) a = cea ps qs (a + (-1) *
                                     p * logBase 2 q)
```

Computing Cross Entropy, take 4

Example (`zip`, `map`, `lambda` and `foldr`)

```
crossEnt ts ps =  
  foldr (+) 0  
    (map (\ (t,p) -> (-1) * t * (logBase 2 p))  
      (zip ts ps))
```

Computing Cross Entropy, take 5

Example (zip, map, lambda and sum)

```
crossEnt ts ps =  
    sum  
        (map (\ (t,p) -> (-1) * t * (logBase 2 p))  
              (zip ts ps))
```

Computing Cross Entropy, take 6

Example (`zipWith`, `lambda` and `sum`)

```
crossEnt ts ps =  
  sum  
    (zipWith (\ t p -> (-1) * t * (logBase 2 p))  
      ts ps)
```

Computing Cross Entropy, take 7

Example (List comprehension)

```
crossEnt ts ps =  
    sum [ (-1) * (ts !! i) * (logBase 2 (ps !! i))  
          | i <- [ 0 .. length ts - 1 ] ]
```

Introducing a new type: Set

How to store set elements

We'll keep sets in form of **lists sorted in ascending order**.

Example (Creating the new type)

```
type Set a = [a]
```

Adding an element to a set

Example

```
adElem :: Ord t => t -> Set t -> Set t
adElem e [] = [e]
adElem e (a:x) | e < a      = e:a:x
                | e == a     = a:x
                | otherwise = a:adElem e x
```

```
Main> adElem 2 [1,3,5]
[1,2,3,5]
```

Homework

Does `adElem` make sure that each element is unique?

Building sets from lists

Example

```
listToSet :: Ord t => [t] -> Set t
listToSet []      = []
listToSet (x:xs) = adElem x (listToSet xs)
```

```
Main> listToSet [1,2,4,3,4,2,1]
[1,2,3,4]
```

Cardinality and set membership

Example

```
card :: Set a -> Int
card s = length s
```

```
inSet :: Eq t => t -> Set t -> Bool
inSet e [] = False
inSet e (x:xs) | e == x = True
                | otherwise = inSet e xs
```

Homework

Could you implement `inSet` in a more efficient fashion?

Eliminating one element from a set

Example

```
delElem :: Ord t => t -> Set t -> Set t
delElem e [] = []
delElem e (a:x) | e < a = a:x
                  | e == a = x
                  | otherwise = a:delElem e x
```

Set union

Example

```
union :: Ord a => Set a -> Set a -> Set a
union [] t      = t
union s []      = s
union (a:s) (b:t)
  | a == b      = a:union s t
  | a < b       = a:union s (b:t)
  | otherwise   = b:union (a:s) t
```

```
Main> union [1,3,4] [2,3,5]
[1,2,3,4,5]
```

Your turn

Homework

Write functions for set intersection and difference.

Set inclusion

Example

```
subset :: Ord a => Set a -> Set a -> Bool
subset [] _      = True
subset _ []      = False
subset (a:s) (b:t)
  | a == b      = subset s t
  | a < b       = False
  | otherwise   = subset (a:s) t
```

Remark

```
Main> subset [] []
True
```

Set equality

Example

```
eqset :: Ord a => Set a -> Set a -> Bool
eqset xs ys | subset xs ys = subset ys xs
             | otherwise    = False
```

Homework

Can you implement `eqset` in a more efficient manner?

All subsets of a set

Example

```
powerset :: Set a -> [Set a]
powerset xs = pSet xs [] where
    pSet [] base      = [base]
    pSet (x:xs) base = pSet xs base ++
                        pSet xs (x:base)

Main> powerset [1,2,3]
[[], [3], [2], [3,2], [1], [3,1], [2,1], [3,2,1]]
```

Cartesian product

Example

```
cartprod :: Set a -> Set b -> [(a,b)]
cartprod []      ys = []
cartprod (x:xs) ys = putx x ys
  where
    putx []      = cartprod xs ys
    putx (z:zs) = (x,z):(putx zs)
```



```
cartprod ["A", "F"] ["Lt", "Ro", "Ser"]
[("A", "Lt"), ("A", "Ro"), ("A", "Ser"),
 ("F", "Lt"), ("F", "Ro"), ("F", "Ser")]
```

Homework

Can you implement `cartprod` in a different manner?

Think! (courtesy of IBM & Auguste Rodin)

THINK



JUNE 1935

Think!

Example (Given the above, what does this function do?)

```
ors s = foldr (++) [] (map perm (powerset s))
```

```
Main> ors [1,2,3]
```

Think!

Example (And how about this one ?)

```
c [] = [[]]
```

```
c (x:xs) = c xs ++ map (x:) (c xs)
```

Example (A trace)

```
c [1,2,3]
```

```
= c [2,3] ++ map (1:) c [2,3]
```

```
= (c [3] ++ map (2:) c [3])
```

```
  ++ map (1:) c [2,3]
```

```
= ((c [] ++ map (3:) c []) ++ map (2:) c [3])
```

```
  ++ map (1:) c [2,3]
```

```
= ((([]) ++ map (3:) []) ++ map (2:) c [3])
```

```
  ++ map (1:) c [2,3]
```

```
= ((([]) ++ [3]) ++ map (2:) c [3])
```

```
  ++ map (1:) c [2,3]
```

Think!

Example (And how about this one ?)

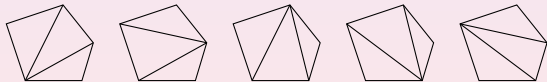
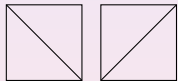
```
c [] = []  
c (x:xs) = c xs ++ map (x:) (c xs)
```

Example (A trace)

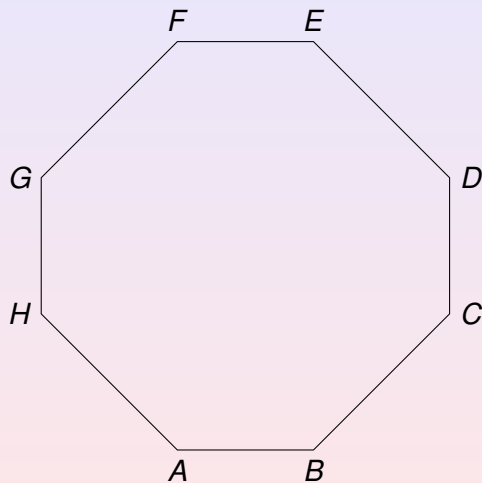
```
=( [], [3] ++ map (2:) [ [], [3] ] )  
  ++ map (1:) c [2,3]  
=( [], [3] ++ [ [2], [2,3] ] )  
  ++ map (1:) c [2,3]  
=[ [], [3], [2], [2,3] ] ++ map (1:) c [2,3]  
=[ [], [3], [2], [2,3] ] ++ map (1:) [ [], [3], [2], [2,3] ]  
=[ [], [3], [2], [2,3] ] ++ [ [1], [1,3], [1,2], [1,2,3] ]  
=[ [], [3], [2], [2,3], [1], [1,3], [1,2], [1,2,3] ]
```

A problem by Euler

In how many ways can we triangulate a convex polygon by non-intersecting diagonals?

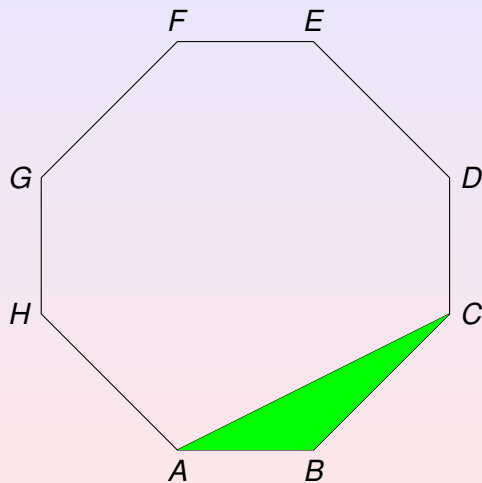


Counting triangulations T_n in a systematic way



$$T_3 = 1, T_4 = 2$$

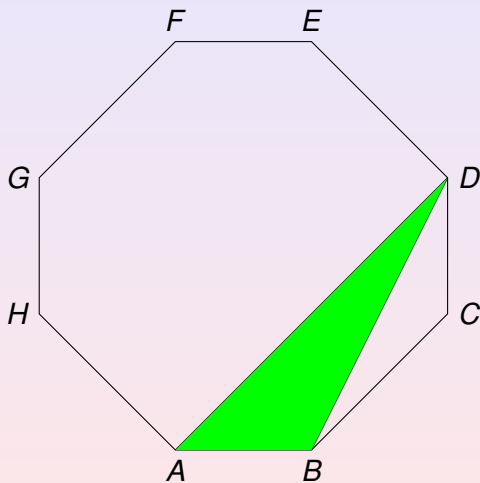
Counting triangulations T_n in a systematic way



$$T_3 = 1, T_4 = 2$$

$$T_8 = T_7 +$$

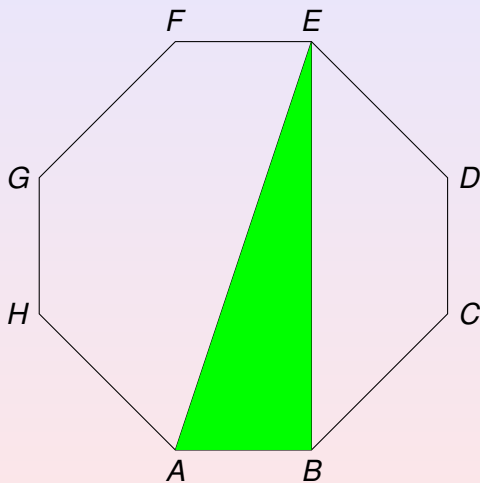
Counting triangulations T_n in a systematic way



$$T_3 = 1, T_4 = 2$$

$$T_8 = T_7 + T_6 * T_3 +$$

Counting triangulations T_n in a systematic way



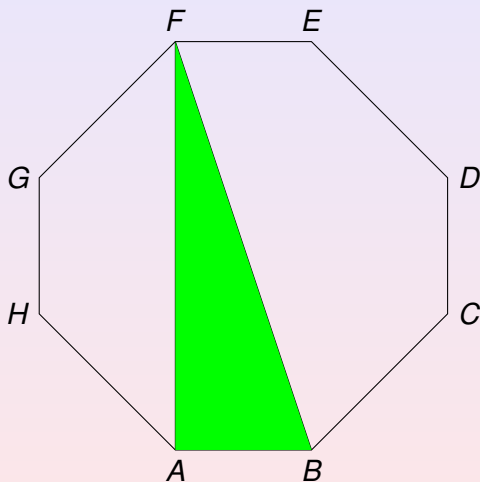
$$T_3 = 1, T_4 = 2$$

$$T_8 = T_7 +$$

$$T_6 * T_3 +$$

$$T_5 * T_4 +$$

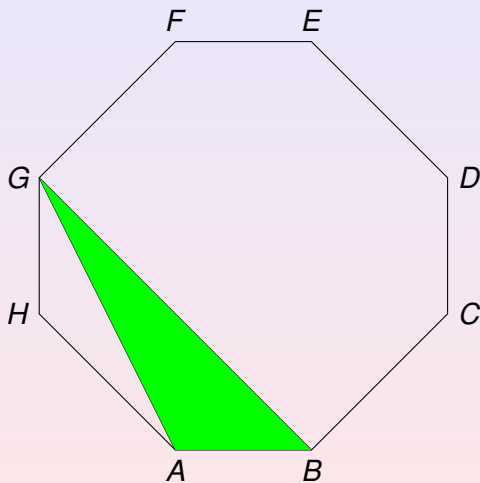
Counting triangulations T_n in a systematic way



$$T_3 = 1, T_4 = 2$$

$$T_8 = T_7 +$$
$$T_6 * T_3 +$$
$$T_5 * T_4 +$$
$$T_4 * T_5 +$$

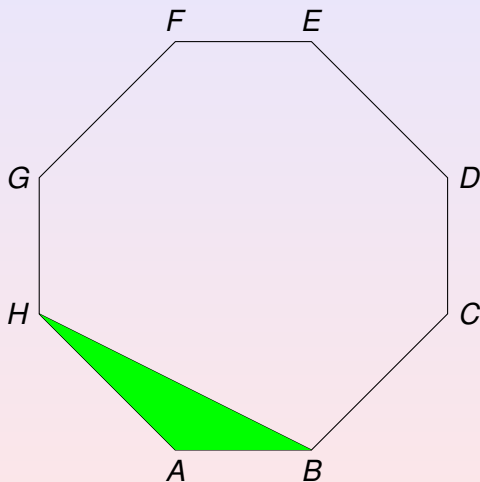
Counting triangulations T_n in a systematic way



$$T_3 = 1, T_4 = 2$$

$$T_8 = T_7 +$$
$$T_6 * T_3 +$$
$$T_5 * T_4 +$$
$$T_4 * T_5 +$$
$$T_3 * T_6 +$$

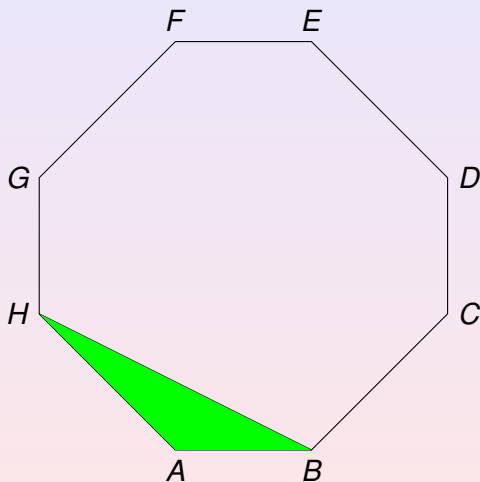
Counting triangulations T_n in a systematic way



$$T_3 = 1, T_4 = 2$$

$$T_8 = T_7 +$$
$$T_6 * T_3 +$$
$$T_5 * T_4 +$$
$$T_4 * T_5 +$$
$$T_3 * T_6 +$$
$$T_7$$

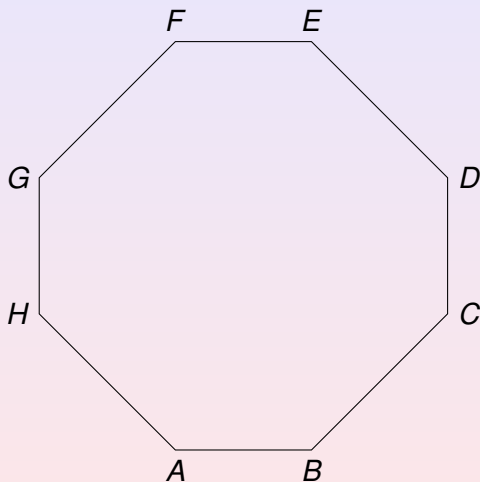
Counting triangulations T_n in a systematic way



$$T_2 = 1, T_3 = 1, T_4 = 2$$

$$T_8 = T_7 * T_2 +$$
$$T_6 * T_3 +$$
$$T_5 * T_4 +$$
$$T_4 * T_5 +$$
$$T_3 * T_6 +$$
$$T_2 * T_7$$

Counting triangulations T_n in a systematic way

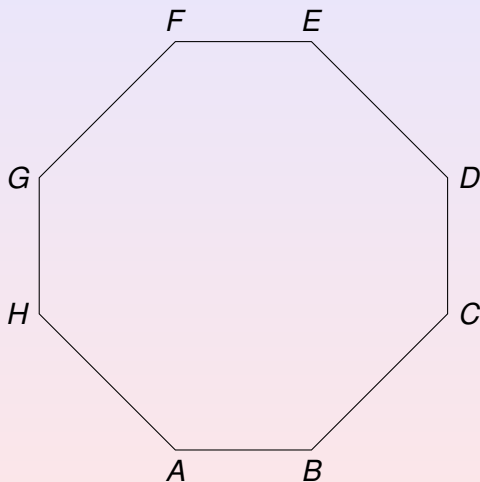


$$T_2 = 1, T_3 = 1, T_4 = 2$$

$$T_8 = T_7 * T_2 + \\ T_6 * T_3 + \\ T_5 * T_4 + \\ T_4 * T_5 + \\ T_3 * T_6 + \\ T_2 * T_7$$

$$T_8 = \sum_{i=2}^{8-1} T_i * T_{8+1-i}$$

Counting triangulations T_n in a systematic way



$$T_2 = 1, T_3 = 1, T_4 = 2$$

$$T_8 = T_7 * T_2 + \\ T_6 * T_3 + \\ T_5 * T_4 + \\ T_4 * T_5 + \\ T_3 * T_6 + \\ T_2 * T_7$$

$$T_8 = \sum_{i=2}^{8-1} T_i * T_{8+1-i}$$

$$T_n = \sum_{i=2}^{n-1} T_i * T_{n+1-i}$$

Computing T_n

Formula

$$T_n = \sum_{i=2}^{n-1} T_i * T_{n+1-i}$$

Haskell code v1: straightforward, but not efficient

```
t1 n | n <= 3      = 1
     | otherwise = sum [(t1 i) * (t1 (n+1-i)) |
                        i <- [2..(n-1)] ]
```

Computing T_n

Formula

$$T_n = \sum_{i=2}^{n-1} T_i * T_{n+1-i}$$

Haskell code v2: more efficient

```
t2 n = t2a 1 [1] where
  t2a k cs
    | k == n-1  = head cs
    | otherwise = t2a (k+1) (cn:cs) where
                        cn = sum (zipWith (*)
                                cs (reverse cs))
```

Using T_n : an example

Multiplying matrices A, B, C, D

Sizes: $A : 2 \times 3$; $B : 3 \times 4$; $C : 4 \times 5$; $D : 5 \times 6$.

We can do $A * (B * (C * D))$ or $((A * B) * C) * D$ or ...

Questions:

1. In how many ways can we perform the operations?
2. How many multiplication ops do we perform in each case?

Fibonacci Numbers

Example (Basic solution)

```
fib n | n==0      = 0
      | n==1      = 1
      | otherwise = fib (n-1) + fib (n-2)
```

Example (Tail recursive, two accumulators)

```
fib n = fib2 n 1 0 where
  fib2 n i j | n==0      = j
              | otherwise = fib2 (n-1) (i+j) i
```

Fibonacci Numbers

Matrix-based computation (Knuth)

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

(D. E. Knuth. *The Art of Computer Programming, 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley, 1997, p. 343)

Fibonacci Numbers

Example (2×2 matrix multiplication)

```
data Matrix2x2 = Mat2 ((Integer, Integer),  
                        (Integer, Integer))  
    deriving (Show)  
  
instance Num Matrix2x2 where  
    Mat2 (a11, a12) * Mat2 (b11, b12) =  
        let  
            (a11, a12) = a11  
            (a21, a22) = a12  
            (b11, b12) = b11  
            (b21, b22) = b12  
        in  
            Mat2 ((a11*b11+a12*b21, a11*b12+a12*b22),  
                  (a21*b11+a22*b21, a21*b12+a22*b22))
```

Fibonacci Numbers

Example (Powers of a 2×2 matrix, linear)

```
infixr 8 |^|  
Mat2 (a11, a12) |^| n  
  | n==0      = Mat2 ((1,0), (0,1))  
  | otherwise = Mat2 (a11, a12) *  
                    Mat2 (a11, a12) |^| (n-1)
```

Fibonacci Numbers

Example (Powers of a 2×2 matrix, logarithmic)

```
infixr 8 |^|  
m |^| n | n == 0 = Mat2 ((1, 0), (0, 1))  
      | even n   = m |^| (n `div` 2) *  
                  m |^| (n `div` 2)  
      | odd  n   = m * m |^| (n `div` 2) *  
                  m |^| (n `div` 2)
```

Fibonacci Numbers

Example (Powers of a 2×2 matrix, logarithmic, tail recursive)

```
infixr 8 |^|  
(|^|) = mpow (Mat2 ((1, 0), (0, 1)))  
  
mpow a m n | n == 0 = a  
           | even n = mpow a  
                       (m * m)  
                       (n `div` 2)  
           | odd n  = mpow (m * a)  
                       (m * m)  
                       (n `div` 2)
```

Fibonacci Numbers

Example (Extracting F_n)

```
fib n = get_fib (kf |^| n) where  
  kf = Mat2 ((1,1), (1,0))  
  get_fib (Mat2 ((_, a12), _)) = a12
```

Derangements

First dance at a prom: ladies' choice

K couples go to a prom. In how many ways can ladies invite gentlemen to dance s.t. no lady dances with her date?

Definition (Derangement)

Permutation with no fix point ($\forall i, i \neq \sigma(i)$)

Recursive formula

$$!n = (n-1) * (! (n-1) + ! (n-2))$$

Derangements

Recursive formula

$!n = (n - 1) * (! (n - 2) + ! (n - 1))$, with $!0 = 1$, $!1 = 0$

Justification

Lady #1 picks partner #k, $k \neq 1$. She has $(n - 1)$ options. Now:

Case A: lady #k invites gentleman #1. We have:

L: 2,3,...,k-1,k+1,...,n

G: 2,3,...,k-1,k+1,...,n

Same problem, dimension $(n - 2)$

Case B: lady #k now has to pick gentleman #j, $j \neq k$, $j \neq 1$:

L: 2,3,...,k-1,**k**,k+1,...,n

G: 2,3,...,k-1,**1**,k+1,...,n

Same problem, dimension **$(n - 1)$**

Derangements

Non-recursive formula: find N s.t. $!n = n! - N$

S_k = set of permutations of n objects that fix object k , $1 \leq k \leq n$

$$\begin{aligned} N &= |S_1 \cup \dots \cup S_n| \\ &= \sum_i |S_i| - \sum_{i < j} |S_i \cap S_j| + \dots + (-1)^{n+1} |S_1 \cap \dots \cap S_n| \\ &= \binom{n}{1}(n-1)! - \binom{n}{2}(n-2)! + \dots + (-1)^{n+1} \binom{n}{n} 0! \\ &= n! \sum_{i=1}^n \frac{(-1)^{i+1}}{i!} \\ \text{So: } !n &= n! \sum_{i=0}^n \frac{(-1)^i}{i!} \end{aligned}$$

Derangements

Approximation

$$!n = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$$

$$\lim_{n \rightarrow \infty} \frac{!n}{n!} = \frac{1}{e}$$

Summary

- ▶ To write functions, we can mix stack and tail recursion, higher order functions and list comprehensions
- ▶ Lists allow implementing and manipulating sets
- ▶ Extra parameters help in caching results in order to avoid repeating computations